RELATÓRIO FINAL — TP2: TRANSFERÊNCIA DE ARQUIVOS PEER-TO-PEER

CEFET-MG · Sistemas Distribuídos · 2026/2

Autoras: Thainá Martins e Giovana Laura

Código-fonte: https://github.com/ThainaCMartins/TP2-Sistemas-Distribuidos

Evidência oficial: https://github.com/ThainaCMartins/TP2-Sistemas-Distribuidos/actions/runs/41076176482

Versão testada: e5c7a90622f4961c9695d21646400065fb893691.

1. Objetivo e arquitetura

Implementou-se uma aplicação P2P de transferência de arquivos em Python, usando TCP e asyncio. Cada peer executa um servidor e conexões clientes concorrentes; não há tracker. Os endereços dos vizinhos e os metadados são conhecidos previamente. A fragmentação é feita em blocos de 1024 ou 4096 bytes. Cada bloco recebido é verificado por SHA-256, armazenado no deslocamento correspondente e pode ser disponibilizado imediatamente a outros peers. O arquivo final só é publicado após a verificação de seu tamanho e hash. Arquivos parciais são revalidados na retomada.

O protocolo usa mensagens HELLO, HELLO_ACK, BITFIELD, GET, BLOCK e ERROR, com enquadramento TCP formado por comprimento do cabeçalho, JSON e payload binário. inflight evita solicitações simultâneas do mesmo bloco a fontes distintas. O projeto contém p2p/, experimentos.py, validar_funcional.py, executar_testes.py e tests/test_core.py.

2. Validação funcional

Teste 1 — Malha completa A/B/C/D. Cada um dos quatro peers possui três vizinhos. A começa com o arquivo completo; B, C e D recebem e podem compartilhar blocos antes de completar. São verificadas as cópias e a ocorrência de retransmissões entre receptores. Após a conclusão, seis sondas independentes realizam pedidos TCP GET e recebem BLOCK, com validação SHA-256, para comprovar transferência efetiva em A–B, A–C, A–D, B–C, B–D e C–D.

Teste 2 — Troca bidirecional. Dois peers iniciam com subconjuntos complementares de blocos, sem um seeder completo, e ambos enviam e recebem até remontar o arquivo.

Teste 3 — Interrupção e retomada. Um receptor é interrompido; um bloco parcial é adulterado; após reiniciar, o armazenamento revalida o conteúdo, preserva os blocos íntegros e substitui o bloco inválido.

Os três testes funcionais passaram, assim como os 13 testes unitários do protocolo e armazenamento, na execução nº 27 do GitHub Actions.

3. Planejamento experimental

Foram cruzados seis tamanhos de arquivo (10 KiB, 20 KiB, 1 MiB, 5 MiB, 10 MiB e 20 MiB), dois tamanhos de bloco (1024 e 4096 bytes) e duas quantidades totais de peers (2 e 4), totalizando 24 cenários. Os arquivos são gerados com bytes aleatórios. A executa como seeder; os demais baixam em paralelo. O ambiente é Ubuntu/Linux com Python 3.12, TCP IPv4 em 127.0.0.1, sem atraso artificial nos experimentos. Cada cenário foi executado uma vez.

A métrica principal é max_download_seconds: o maior tempo entre o evento READY e a conclusão dos receptores daquele cenário. Os arquivos finais são conferidos externamente por tamanho e SHA-256; os logs permitem auditar blocos enviados, recebidos e suas fontes. A execução validada terminou com 24/24 cenários OK.

4. Resultados

Arquivo	2 peers, 1024 B	2 peers, 4096 B	4 peers, 1024 B	4 peers, 4096 B
10 KiB	0,093	0,094	0,023	0,086
20 KiB	0,012	0,093	0,023	0,012
1 MiB	0,827	0,228	1,085	0,248
5 MiB	11,150	1,347	13,797	1,559

Arquivo	2 peers, 1024 B	2 peers, 4096 B	4 peers, 1024 B	4 peers, 4096 B
10 MiB	40,256	3,588	50,725	4,500
20 MiB	161,365	11,272	194,339	13,820

Tempos em segundos, extraídos do log da execução validada nº 27.

Para 20 MiB, com dois peers, a troca de blocos de 1024 B por 4096 B reduziu o tempo de 161,365 s para 11,272 s (aproximadamente 14,3 vezes menor). Com quatro peers, a redução foi de 194,339 s para 13,820 s (aproximadamente 14,1 vezes menor). Isso é compatível com menor quantidade de pedidos, respostas, verificações e registros de blocos. Nos cenários maiores, quatro peers não foram mais rápidos do que dois, apesar da redistribuição entre receptores; há sobrecarga de concorrência e compartilhamento de recursos do mesmo host.

Os gráficos correspondentes são apresentados no PDF final. Arquivos de até 20 KiB apresentam diferenças pequenas e instáveis, não adequadas a conclusões gerais sobre desempenho.

5. Conclusão e limitações

A implementação atendeu aos testes unitários, funcionais e experimentais executados. Os 24 cenários terminaram com integridade verificada, e a malha completa foi comprovada por seis transferências TCP adicionais com SHA-256. Blocos maiores foram significativamente mais eficientes nos arquivos maiores deste ambiente. Os resultados são restritos a uma repetição por cenário, TCP em loopback e execução em um único computador; não representam estimativas para uma rede física ou para outros equipamentos. A integridade por SHA-256 também não implica autenticação de peers nem confidencialidade dos dados.

Fontes primárias: código no repositório; log e artefato evidencias-execucao-p2p da [execução nº 27](https://github.com/ThainaCMartins/TP2-Sistemas-Distribuidos/actions/runs/41076176482), commit e5c7a906.